

Отчёт по лабораторной работе №12

Программирование в командном процессоре ОС UNIX. Командные файлы

Терентеевская Светлана Александровна

2026-04-09

Содержание I

- 1 Информация
- 2 Вводная часть
- 3 Выполнение лабораторной работы
- 4 Выводы
- 5 Список литературы

Раздел 1

Информация

- Терентеевская Светлана Александровна

- Терентеевская Светлана Александровна
- Группа НКАбд-05-25, студент бакалавра

- Терентеевская Светлана Александровна
- Группа НКАбд-05-25, студент бакалавра
- Российский университет дружбы народов им. П. Лумумбы

- Терентеевская Светлана Александровна
- Группа НКАбд-05-25, студент бакалавра
- Российский университет дружбы народов им. П. Лумумбы
- 1032253498@rudn.ru

- Терентеевская Светлана Александровна
- Группа НКАбд-05-25, студент бакалавра
- Российский университет дружбы народов им. П. Лумумбы
- 1032253498@rudn.ru
- <https://github.com/SvetlanaTerenteevskaya>

Раздел 2

Вводная часть

Цель работы

Изучить основы программирования в оболочке ОС UNIX/Linux. Научиться писать небольшие командные файлы.

- Сделать резервную копию самого скрипта в каталог `~/backup` с использованием архиватора.

- Сделать резервную копию самого скрипта в каталог `~/backup` с использованием архиватора.
- Обработать произвольное число аргументов командной строки.

- Сделать резервную копию самого скрипта в каталог `~/backup` с использованием архиватора.
- Обработать произвольное число аргументов командной строки.
- Вывести содержимое каталога и права доступа к файлам без использования `ls` и `dir`.

Стандартные переменные

Переменная	Назначение
<code>PATH</code>	Список каталогов для поиска команд (разделитель — двоеточие). По умолчанию: текущий каталог, <code>/bin</code> , <code>/usr/bin</code>
<code>HOME</code>	Домашний каталог пользователя
<code>IFS</code>	Разделители полей в командной строке (пробел, табуляция, перевод строки)
<code>MAIL</code>	Файл для проверки почты
<code>TERM</code>	Тип используемого терминала
<code>LOGNAME</code>	Регистрационное имя пользователя

Метасимволы и экранирование
Метасимволы — символы, имеющие специальный смысл: * ? [] > < | & \ \$; # ` () { } ' " .

Назначение основных метасимволов:

- * — любая строка (в том числе пустая)

Экранирование — снятие специального смысла:

Метасимволы и экранирование
Метасимволы — символы, имеющие специальный смысл: * ? [] > < | & \ \$; # ` () { } ' " .

Назначение основных метасимволов:

- * — любая строка (в том числе пустая)
- ? — один любой символ

Экранирование — снятие специального смысла:

Метасимволы и экранирование
Метасимволы — символы, имеющие специальный смысл: * ? [] > < | & \ \$; # ` () { } ' " .

Назначение основных метасимволов:

- * — любая строка (в том числе пустая)
- ? — один любой символ
- [c1-c2] — любой символ из диапазона

Экранирование — снятие специального смысла:

Метасимволы и экранирование
Метасимволы — символы, имеющие специальный смысл: * ? [] > < | & \ \$; # ` () { } ' " .

Назначение основных метасимволов:

- * — любая строка (в том числе пустая)
- ? — один любой символ
- [c1-c2] — любой символ из диапазона

Экранирование — снятие специального смысла:

- Обратный слеш перед метасимволом

Метасимволы и экранирование
Метасимволы — символы, имеющие специальный смысл: * ? [] > < | & \ \$; # ` () { } ' " .

Назначение основных метасимволов:

- * — любая строка (в том числе пустая)
- ? — один любой символ
- [c1-c2] — любой символ из диапазона

Экранирование — снятие специального смысла:

- Обратный слеш перед метасимволом
- Одинарные кавычки (полное экранирование)

Метасимволы и экранирование
Метасимволы — символы, имеющие специальный смысл: * ? [] > < | & \ \$; # ` () { } ' " .

Назначение основных метасимволов:

- * — любая строка (в том числе пустая)
- ? — один любой символ
- [c1-c2] — любой символ из диапазона

Экранирование — снятие специального смысла:

- Обратный слеш перед метасимволом
- Одинарные кавычки (полное экранирование)
- Двойные кавычки (частичное экранирование)

Командные файлы

Командный файл (скрипт) — текстовый файл с последовательностью команд. Для запуска необходимо установить право на выполнение и указать путь к файлу.

Функции

Функция определяется один раз в начале скрипта и может вызываться по имени.

Специальные переменные для командных файлов

Переменная	Назначение
\$0	Имя скрипта
\$1...\$9	Аргументы командной строки
\$#	Количество аргументов
\$@	Все аргументы как отдельные слова
\$*	Все аргументы как одна строка
\$?	Код завершения последней выполненной команды
\$\$	PID текущего процесса
\$!	PID последнего фонового процесса
\${#name}	Длина строки в переменной name
\${name:-value}	Если переменная не определена, подставляется значение
\${name:=value}	ей присваивается значение

Команда getopt

`getopt` осуществляет синтаксический анализ командной строки, выделяя флаги (опции). Строка опций содержит список допустимых флагов; если после флага ожидается аргумент, ставится двоеточие. `getopt` использует переменные `OPTARG` (аргумент флага) и `OPTIND` (индекс следующего аргумента). Обычно `getopt` применяется в цикле `while` совместно с оператором `case`.

Управляющие конструкции

Цикл for

Перебирает элементы списка, последовательно присваивая их переменной и выполняя заданные команды.

Оператор выбора case

Сравнивает значение переменной с несколькими шаблонами и выполняет команды, соответствующие первому совпавшему шаблону. Метасимвол * используется как шаблон по умолчанию.

Условный оператор if

Выполняет проверку условия. Если последняя команда в списке после `if` возвращает нулевой код (истина), выполняется блок `then`. Если нет — проверяется `elif` или выполняется блок `else`. Завершается конструкция служебным словом `fi`.

Команда `test` используется для проверки условий:

- `-d` — проверка, является ли файл каталогом

Цикл while

Выполняется, пока последняя команда в списке условий возвращает нулевой код (истина).

Условный оператор if

Выполняет проверку условия. Если последняя команда в списке после `if` возвращает нулевой код (истина), выполняется блок `then`. Если нет — проверяется `elif` или выполняется блок `else`. Завершается конструкция служебным словом `fi`.

Команда `test` используется для проверки условий:

- `-d` — проверка, является ли файл каталогом
- `-f` — проверка, является ли файл обычным файлом

Цикл while

Выполняется, пока последняя команда в списке условий возвращает нулевой код (истина).

Условный оператор if

Выполняет проверку условия. Если последняя команда в списке после `if` возвращает нулевой код (истина), выполняется блок `then`. Если нет — проверяется `elif` или выполняется блок `else`. Завершается конструкция служебным словом `fi`.

Команда `test` используется для проверки условий:

- `-d` — проверка, является ли файл каталогом
- `-f` — проверка, является ли файл обычным файлом
- `-r` — проверка доступа на чтение

Цикл while

Выполняется, пока последняя команда в списке условий возвращает нулевой код (истина).

Условный оператор if

Выполняет проверку условия. Если последняя команда в списке после `if` возвращает нулевой код (истина), выполняется блок `then`. Если нет — проверяется `elif` или выполняется блок `else`. Завершается конструкция служебным словом `fi`.

Команда `test` используется для проверки условий:

- `-d` — проверка, является ли файл каталогом
- `-f` — проверка, является ли файл обычным файлом
- `-r` — проверка доступа на чтение
- `-w` — проверка доступа на запись

Цикл while

Выполняется, пока последняя команда в списке условий возвращает нулевой код (истина).

Условный оператор if

Выполняет проверку условия. Если последняя команда в списке после `if` возвращает нулевой код (истина), выполняется блок `then`. Если нет — проверяется `elif` или выполняется блок `else`. Завершается конструкция служебным словом `fi`.

Команда `test` используется для проверки условий:

- `-d` — проверка, является ли файл каталогом
- `-f` — проверка, является ли файл обычным файлом
- `-r` — проверка доступа на чтение
- `-w` — проверка доступа на запись
- `-x` — проверка доступа на выполнение

Цикл while

Выполняется, пока последняя команда в списке условий возвращает нулевой код (истина).

Цикл until

Выполняется, пока последняя команда в списке условий возвращает ненулевой код (ложь). Условие выхода из цикла —противоположное while.

Команды true и false

- true — всегда возвращает нулевой код (истина)

Прерывание циклов

Цикл until

Выполняется, пока последняя команда в списке условий возвращает ненулевой код (ложь). Условие выхода из цикла —противоположное while.

Команды true и false

- true — всегда возвращает нулевой код (истина)
- false — всегда возвращает ненулевой код (ложь)

Прерывание циклов

Цикл until

Выполняется, пока последняя команда в списке условий возвращает ненулевой код (ложь). Условие выхода из цикла —противоположное while.

Команды true и false

- true — всегда возвращает нулевой код (истина)
- false — всегда возвращает ненулевой код (ложь)

Прерывание циклов

- break — полностью завершает выполнение цикла

Цикл until

Выполняется, пока последняя команда в списке условий возвращает ненулевой код (ложь). Условие выхода из цикла —противоположное while.

Команды true и false

- true — всегда возвращает нулевой код (истина)
- false — всегда возвращает ненулевой код (ложь)

Прерывание циклов

- break — полностью завершает выполнение цикла
- continue — завершает текущую итерацию и переходит к следующей

Раздел 3

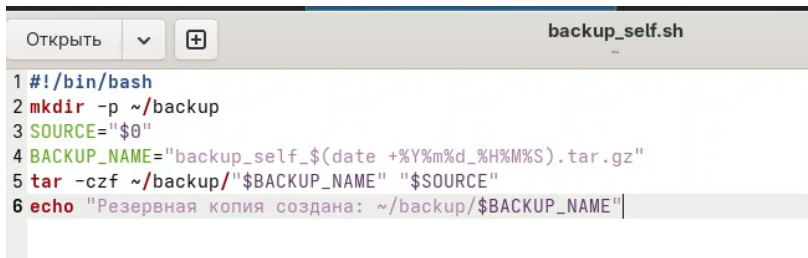
Выполнение лабораторной работы

- 1 Я создала и открыла файл backup_self.sh (рис. 1).

```
saterenteevskaya@saterenteevskaya:~$ cd  
saterenteevskaya@saterenteevskaya:~$ gedit backup_self.sh
```

Рисунок 1: Открывание файла

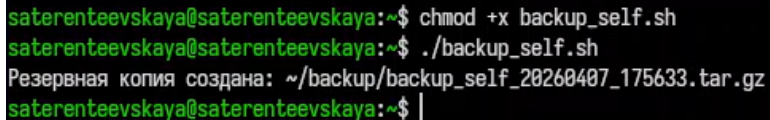
Написала в нем скрипт, который при запуске будет делать резервную копию самого себя (то есть файла, в котором содержится его исходный код) в другую директорию backup в моём домашнем каталоге (рис. 2).

A screenshot of a terminal window with a light gray title bar. The title bar contains three buttons on the left: 'Открыть', a dropdown arrow, and a '+' icon. On the right side of the title bar, the text 'backup_self.sh' is displayed. The terminal area has a white background and contains six lines of code, each preceded by a line number from 1 to 6. The code is as follows:

```
1 #!/bin/bash
2 mkdir -p ~/backup
3 SOURCE="$0"
4 BACKUP_NAME="backup_self_$(date +%Y%m%d_%H%M%S).tar.gz"
5 tar -czf ~/backup/"$BACKUP_NAME" "$SOURCE"
6 echo "Резервная копия создана: ~/backup/$BACKUP_NAME"
```

Рисунок 2: Скрипт для backup_self.sh

Запустила (рис. 3).

A terminal window with a black background and green text. The prompt is 'saterenteevskaya@saterenteevskaya:~\$'. The first command is 'chmod +x backup_self.sh'. The second command is './backup_self.sh'. The output is 'Резервная копия создана: ~/backup/backup_self_20260407_175633.tar.gz'. The prompt is followed by a vertical bar '|'.

```
saterenteevskaya@saterenteevskaya:~$ chmod +x backup_self.sh
saterenteevskaya@saterenteevskaya:~$ ./backup_self.sh
Резервная копия создана: ~/backup/backup_self_20260407_175633.tar.gz
saterenteevskaya@saterenteevskaya:~$ |
```

Рисунок 3: Запуск backup_self.sh

Проверила, что создался архив (2 раза) (рис. 4).

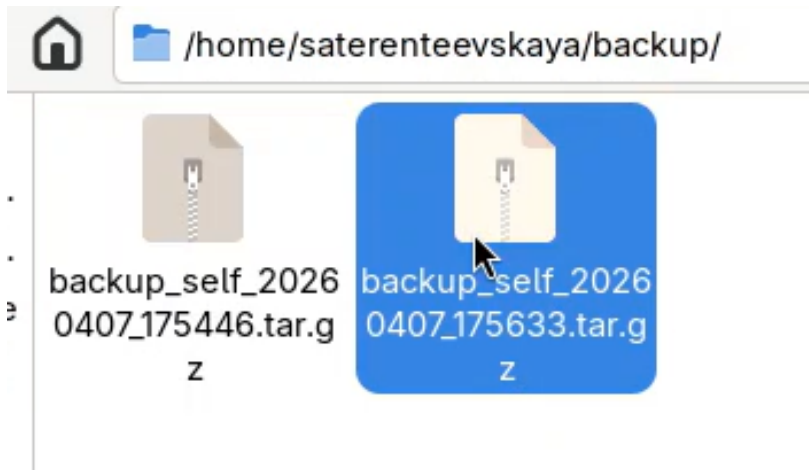


Рисунок 4: Проверка архива

- 2 Я написала пример командного файла, обрабатывающего любое произвольное число аргументов командной строки, в том числе превышающее десять (рис. 5).



```
1 #!/bin/bash
2 echo "Всего аргументов: $#"
```

```
3 counter=1
4 for arg in "$@"; do
5 echo "Аргумент №$counter: $arg"
6 counter=$((counter + 1))
7 done
8
```

Рисунок 5: Скрипт для args.sh

Запустила с разными аргументами (рис. 6).

```
saterenteevskaya@saterenteevskaya:~$ chmod +x args.sh
saterenteevskaya@saterenteevskaya:~$ ./args.sh
Всего аргументов: 0
saterenteevskaya@saterenteevskaya:~$ ./args.sh 1 2 3 4 5 6 7 8 9 10 11
Всего аргументов: 11
Аргумент №1: 1
Аргумент №2: 2
Аргумент №3: 3
Аргумент №4: 4
Аргумент №5: 5
Аргумент №6: 6
Аргумент №7: 7
Аргумент №8: 8
Аргумент №9: 9
Аргумент №10: 10
Аргумент №11: 11
saterenteevskaya@saterenteevskaya:~$ ./args.sh a s f g h y r f v
Всего аргументов: 9
Аргумент №1: a
Аргумент №2: s
Аргумент №3: f
Аргумент №4: g
Аргумент №5: h
Аргумент №6: y
Аргумент №7: r
Аргумент №8: f
Аргумент №9: v
```

Рисунок 6: Запуск args.sh

- 3 Я написала командный файл — аналог команды `ls` (без использования самой этой команды и команды `dir`). Он выдает информацию о нужном каталоге и выводит информацию о возможностях доступа к файлам этого каталога (рис. 7, рис. 8)

```
gedit myls.sh      myls.sh (-) - gedit      backu
Открыть  ▼  +  myls.sh
~
1 #!/bin/bash
2 if [ -n "$1" ]; then
3     TARGET_DIR="$1"
4 else
5     TARGET_DIR="."
6 fi
7 if [ ! -d "$TARGET_DIR" ]; then
8     echo "Ошибка: '$TARGET_DIR' не является каталогом"
9     exit 1
10 fi
11 for file in "$TARGET_DIR"/*; do
12     if [ -e "$file" ]; then
13         if [ -d "$file" ]; then
14             type="d"
15         else
16             type="-"
17         fi
18         if [ -r "$file" ]; then
19             perm_r="r"
20         else
21             perm_r="-"
22         fi
23         if [ -w "$file" ]; then
24             perm_w="w"
25         else
26             perm_w="-"
27         fi
28         if [ -x "$file" ]; then
29             perm_x="x"
30         else
31             perm_x="-"
```

Рисунок 7: Скрипт для myls.sh

```
28     if [ -x "$file" ]; then
29         perm_x="x"
30     else
31         perm_x="-"
32     fi
33     echo "$type$perm_r$perm_w$perm_x ${basename "$file"}"
34 fi
35 done
```

Рисунок 8: Скрипт для myls.sh

Запустила (рис. 9).

```
saterenteevskaya@saterenteevskaya:~$ chmod +x myls.sh
saterenteevskaya@saterenteevskaya:~$ ./mysls.sh
-rw- archiv.zip
-rwx args.sh
drwx backup
-rwx backup_self.sh
drwx bin
-rw- COURSE
drwx Documents
drwx Downloads
-rw- file1.txt
-rw- file2.txt
-rw- file3.txt
-rw- file4.txt
-rwx format.sh
drwx git-extended
dr-x gitflow
-rw- lab05_Терентеевская_архив.zip
-rw- lab11.sh
-rw- lab11.sh~
-rw- LICENSE
-rwx myls.sh
-rw- PkgTTC-Iosevka-34.2.1.zip
-rw- sway.log
drwx work
drwx Видео
drwx Документы
drwx Загрузки
drwx Изображения
drwx Музыка
drwx Общедоступные
drwx Рабочий стол
```

Рисунок 9: Запуск myls.sh

- 4 Я написала командный файл, который получает в качестве аргумента командной строки формат файла (.txt, .doc, .jpg, .pdf и т.д.) и вычисляет количество таких файлов в указанной директории. Путь к директории также передаётся в виде аргумента командной строки (рис. 10).

Рисунок 10: Скрипт для format.sh

Запустила и проверила с txt (рис. 11).

```
saterenteevskaya@saterenteevskaya:~$ ./format.sh txt ~  
Директория: /home/saterenteevskaya  
Расширение: .txt  
Количество файлов: 4
```

Рисунок 11: Количество txt-файлов

и sh (рис. 12).

```
saterenteevskaya@saterenteevskaya:~$ ./format.sh sh .  
Директория: .  
Расширение: .sh  
Количество файлов: 5
```

Рисунок 12: Количество sh-файлов

Раздел 4

Выводы

В ходе данной лабораторной работы я изучила основы программирования в оболочке ОС UNIX/Linux и научилась писать небольшие командные файлы.

Раздел 5

Список литературы

ТУИС
Методичка к лабораторной работе №12